

Artificial Intelligence Fundamentals.
Lab 1: Implementation of search algorithms.
Master in Artificial Intelligence 2026-2027

Alessia Sedlmayer and Clara Mingyue López Piñeiro

October 2026

Contents

1	Methods	3
1.1	Problem definitions	3
1.1.1	Definitions of state and orientation	3
1.1.2	Movements for the current state	4
1.1.3	Transition model	4
1.1.4	Initial and Goal test	5
1.1.5	Cost function	5
1.2	Blind Search Algorithms	6
1.3	Heuristic function	7
2	Results	8
3	Discussion	9
3.1	Advantages and Disadvantages	9
3.2	Performance of the implemented heuristic	10
3.3	Variation of the original problem	10
4	Appendices	11

1 Methods

1.1 Problem definitions

The predefined problem can be described as a search problem with a finite two-dimensional grid. The goal is to find a sequence of valid movements that navigate from a given state to a goal state. In addition to its position on the map, the orientation is also important since it determines the direction of movement.

1.1.1 Definitions of state and orientation

A state is represented by a tuple

$$s = (r, c, o),$$

where r is the row, c the column and o the current orientation of the state, on the map. The state is implemented using the `State` class shown in Listing 1.

Listing 1: Class for state definition

```
1 @dataclass(frozen=True)
2 class State:
3     row: int
4     column: int
5     orientation: int
```

The orientation of the state is important because it determines the direction of movement when moving forward.

The orientation can have up to eight possible regular values it can take, that correspond with the eight surrounding fields on the grid. These directions are placed clockwise from the state starting from the north, as shown in Table 1.

index	orientation
0	N
1	NE
2	E
3	SE
4	S
5	SW
6	W
7	NW

Table 1: Overview of index corresponding orientations

The value $o = 8$ is a special case where the orientation of the state is not of importance. For the goal state this means, that goals can be reached in the orientation of arrival and do not to be turned around anymore.

1.1.2 Movements for the current state

From each state, up to three different actions can be performed: `move_forward`, `turn_right`, and `turn_left`. The `move_forward` action moves the robot one cell in the direction of its current orientation, while `turn_right` and `turn_left` modify the robot's orientation by rotating it 45° clockwise or counterclockwise, respectively. Consequently, the maximum branching factor of the search problem is three.

The possible successor states are generated by the `successors` function shown in Listing 3 of the Appendices. Since the map has finite boundaries, the `move_forward` action is only considered valid if the resulting coordinates remain within the limits of the grid. Therefore, depending on the current position of the robot, a state may have either two or three valid successors.

1.1.3 Transition model

The transition model formally defines the state resulting from the application of each available action. Let a state be represented as previously defined

$$s = (r, c, o),$$

where r and c denote the current row and column of the robot, and o its orientation.

For each orientation o , a displacement $(\Delta r_o, \Delta c_o)$ is associated with the corresponding direction. Based on this displacement, the transition produced by the `move_forward` action is defined as

$$T((r, c, o), \text{move_forward}) = (r + \Delta r_o, c + \Delta c_o, o).$$

This transition is valid only if the resulting coordinates remain inside the map. For a grid with N rows and M columns, the corresponding precondition is

$$0 \leq r + \Delta r_o < N$$

$$0 \leq c + \Delta c_o < M.$$

If these conditions are not satisfied, the transition is not generated. Notice that the orientation o remains unchanged during this transition.

The transitions corresponding to the two rotation actions are defined as

$$T((r, c, o), \text{turn_right}) = (r, c, (o + 1) \bmod 8),$$

$$T((r, c, o), \text{turn_left}) = (r, c, (o - 1) \bmod 8).$$

Since these transitions only modify the orientation component of the state, they do not require any additional positional precondition and are valid for every state of the grid. The modulo operation ensures the cyclic representation of the eight possible orientations, so that a rotation from orientation 7 to the right results in orientation 0, and a rotation from orientation 0 to the left results in orientation 7.

Therefore, the transition model is deterministic, since for every state and valid action there is exactly one resulting successor state.

1.1.4 Initial and Goal test

The search starts from a predefined initial state and aims to reach a goal state. After each movement with `successors` the new states, are compared with the goal state with the `is_goal` method shown in Listing 2

Listing 2: Check if the current state correspond to the goal state

```

1 def is_goal(self, state: State) -> bool:
2     # The goal can require a specific direction or
      accept any direction.
3     goal_position = state.row == self.goal.row and state
      .column == self.goal.column
4     goal_direction = self.goal.orientation == 8 or state
      .orientation == self.goal.orientation
5     return goal_position and goal_direction

```

Two parts of the state are checked separately, first in line 3 we look if the position on the map is correct and afterwards, in line 4, the orientation is checked. This separated check of orientation is important, since we could have the case if *orientation* = 8 and therefore do not have to worry about the states orientation. For the case that we have a orientation, we have to check it like a normal part of the state.

1.1.5 Cost function

The cost of a path is defined as the sum of the costs of all actions performed. Each 45° rotation, either clockwise or counterclockwise, has a cost of one unit. In contrast, the cost of moving forward depends on the hardness value of the destination cell. Therefore, if the robot moves from state s to a cell (r', c') , the corresponding action cost is $Grid[r'][c']$. Consequently, path depth and path cost are not necessarily equivalent: two paths containing the same number of actions may have very different total costs.

1.2 Blind Search Algorithms

Two blind search strategies are considered for solving the problem: Breadth-First Search (BFS) and Depth-First Search (DFS). Both algorithms explore the state space without using any information about the location of the goal beyond the goal test. However, they differ in the order in which nodes are expanded and, consequently, in their expected execution time, memory requirements, completeness, and optimality.

In the proposed problem, each state has a maximum branching factor of $b = 3$, since the robot can generate at most three successors: moving forward, turning right, or turning left. Furthermore, for a map of N rows and M columns, the number of different states is bounded by

$$|S| \leq 8NM$$

Since each position can be associated with eight possible orientations. Therefore, the search space is finite, although its size increases proportionally with the size of the map.

Breadth-First Search explores the search tree level by level. All nodes at depth d are expanded before nodes at depth $d + 1$. The main disadvantage of BFS is therefore its memory consumption, since it must keep all generated but not yet expanded nodes in the frontier. As the depth of the search increases, the number of stored nodes can grow considerably.

Depth-First Search follows one branch of the search tree as deeply as possible before backtracking and exploring alternative branches. Its memory requirement is considerably lower than that of BFS, since it mainly needs to store the nodes belonging to the current path together with the unexplored alternatives.

Regarding optimality, DFS does not guarantee an optimal solution, either in terms of depth or path cost. BFS guarantees finding a solution with minimum depth, that is, a solution containing the minimum number of actions. However, this does not imply minimum path cost in the proposed problem. The actions do not have uniform costs: each rotation has a cost of one unit, while the cost of moving forward depends on the hardness of the destination cell. Consequently, a path with fewer actions may have a higher total cost than a longer path crossing softer terrain. Therefore, BFS is not guaranteed to be optimal with respect to the accumulated cost $g(n)$.

Considering these properties, BFS is considered *a priori* more suitable than DFS for the proposed problem. Although BFS requires more memory, it systematically explores states in increasing order of depth and guarantees finding a shallowest solution, while DFS may spend a significant amount of time exploring deep branches that lead to solutions of poor quality. Since the state space

is finite and relatively bounded by the dimensions of the map, the additional memory requirements of BFS can be considered acceptable.

1.3 Heuristic function

A heuristic function $h(n)$ estimates the minimum cost of reaching a goal from a given state n . Heuristic functions provide additional information to guide the search towards the most promising states.

For each state, A* applies the evaluation function

$$f(n) = g(n) + h(n),$$

where $f(n)$ represents the estimated total cost of a solution through n , $g(n)$ is the actual accumulated cost from the initial state to n , and $h(n)$ is the estimated cost from n to the goal.

For this problem, we decided to use the Chebyshev distance as the heuristic function, which can be described as

$$h(n) = \max(|x_g - x_n|, |y_g - y_n|),$$

where (x_g, y_g) represents the goal position and (x_n, y_n) represents the position of the current state n .

For the definition of this heuristic, we consider a simplified version of the original problem. In this simplification, every forward movement is assumed to have the minimum possible cost, which is one unit, and the additional cost of the rotations required to orient the robot is ignored. The Chebyshev distance then represents the minimum number of forward movements required to reach the goal when movement in the eight possible directions is considered.

This relaxation provides an optimistic estimation of the real cost. In the original problem, every forward movement has a cost equal to the hardness of the destination cell, which is always greater than or equal to one, and rotations also have an additional cost of one unit. Therefore, the heuristic can never overestimate the actual minimum remaining cost:

$$h(n) \leq h^*(n),$$

where $h^*(n)$ represents the real optimal cost from n to the goal. Consequently, the proposed heuristic is admissible.

The heuristic also satisfies the **consistency** or **monotonicity** property for every successor state n' of n :

$$h(n) \leq c(n, n') + h(n').$$

For rotation actions, the position of the robot does not change, and therefore $h(n) = h(n')$. Since a rotation has cost 1, the consistency condition is satisfied. For a forward movement, the Chebyshev distance can decrease by at most one unit, while the real cost of the movement is always at least one. Therefore, the consistency condition is also satisfied for forward movements.

Thus, the heuristic is monotone. Besides implying admissibility when $h(\text{goal}) = 0$, monotonicity is particularly desirable in A* graph search because it guarantees that

$$f(n) = g(n) + h(n)$$

does not decrease along a path. As a consequence, once a node has been expanded with its best cost, it does not need to be reopened.

2 Results

The results in the following were calculated with experimental maps generated from random values in a predefined size. From the 9x9 maps, the lowest rows and rightest columns were deleted to shorten the maps to the corresponding sizes.

For each table we generated five individual tables and averaged them per category and map size.

In the following we stated four variables **d**, **g**, **#E**, **#F** that were all calculated in the same way.

Predefined it is stated that **d** is the depth of the exploration tree-graph where the solution was found and **g** the cost of the solution. **#E** and **#F** are stating the total number of explored nodes and the final number of nodes stored in the frontier at the end of the execution.

	d	g	#E	#F
Breadth-first	5	7,6	19	9
Depth-first	8	12,8	9	8
A*(h)	5,4	7,4	15,6	8,8

Table 2: Comparison of the performance of the different implemented methods (3x3)

	d	g	#E	#F
Breadth-first	7	11,6	60	32
Depth-first	12	20,2	13	16
A*(h)	7,4	11,2	44,2	33,6

Table 3: Comparison of the performance of the different implemented methods (5x5)

	d	g	#E	#F
Breadth-first	9	16	156	62
Depth-first	16	34,2	17	24
A*(h)	10,2	15	96,2	62,2

Table 4: Comparison of the performance of the different implemented methods (7x7)

	d	g	#E	#F
Breadth-first	11	21,2	316	92
Depth-first	20	41	21	32
A*(h)	12,2	19,6	209,6	111,8

Table 5: Comparison of the performance of the different implemented methods (9x9)

Since those tables only show the averages for each field, it is mentionable that for depth- and breadth-first the values for d, #E and #F were for each individual setting the same.

3 Discussion

3.1 Advantages and Disadvantages

The experimental results show the expected differences between the three search methods. BFS consistently finds the shallowest solutions, but its main drawback is the large number of expanded nodes and the resulting memory consumption. In the 9×9 maps, for example, BFS expands on average 316 nodes.

DFS requires considerably less memory and expands fewer nodes (20 in the same case), but the quality of its solutions is clearly worse. It does not guarantee minimum depth or minimum cost, obtaining an average cost of 41 for the 9×9 maps.

A* offers the best compromise. It obtains the lowest solution cost in all tested

map sizes while expanding fewer nodes than BFS. For the 9×9 case, A* obtains $g = 19.6$ and expands 209.6 nodes, compared with $g = 21.2$ and 316 nodes for BFS. Therefore, A* is the most suitable method when minimizing drilling cost is the main objective.

3.2 Performance of the implemented heuristic

The selected heuristic, the Chebyshev distance, represents the minimum number of forward movements required to reach the goal when rotation costs and terrain hardness are ignored. Since every real movement costs at least one unit, the heuristic never overestimates the real remaining cost and is therefore admissible. It is also consistent, since a rotation does not change its value and a forward movement can reduce it by at most one unit.

The results show that the heuristic effectively guides A*, reducing the number of expanded nodes with respect to BFS while obtaining lower-cost solutions. However, it ignores both the current orientation and the hardness of future cells, so its estimate is relatively simple and could potentially be improved.

3.3 Variation of the original problem

Unbreakable rocks could be represented by a special value in the map indicating a blocked cell. In that case, the `move_forward` action would only be generated if the destination cell is inside the map and is not blocked. Rotation actions would remain unchanged. The maximum branching factor would still be three, although obstacles could reduce the effective number of available actions and could make some goals unreachable.

The Chebyshev heuristic would remain admissible and consistent, since obstacles can only maintain or increase the real cost of reaching the goal. Nevertheless, it would become less informative because it does not consider the detours caused by obstacles.

If a valid path exists, BFS would still find a minimum-depth solution, although not necessarily a minimum-cost one, while DFS would still provide no optimality guarantee. A*, using the proposed admissible and consistent heuristic, would still find an optimal minimum-cost solution. If the goal becomes unreachable, the algorithms will eventually exhaust the reachable state space without finding a solution.

4 Appendices

Listing 3: Check for the possible actions of movement and updating of the steps

```
1 def successors(self, node: Node) -> list[Node]:
2     # Generate the possible actions in this order:
3     # forward, right, left.
4     state = node.state
5     children = []
6
7     row_change, column_change = DIRECTIONS[state.
8     orientation]
9     next_row = state.row + row_change
10    next_column = state.column + column_change
11
12    if self.within_bounds(next_row, next_column):
13        forward_state = State(next_row, next_column,
14        state.orientation)
15        move_cost = self.grid[next_row][next_column]
16        children.append(
17            Node(forward_state, node, "move_forward",
18            node.depth + 1, node.cost + move_cost)
19        )
20
21    right_state = State(state.row, state.column, (state.
22    orientation + 1) % 8)
23    children.append(Node(right_state, node, "turn_right",
24    , node.depth + 1, node.cost + 1))
25
26    left_state = State(state.row, state.column, (state.
27    orientation - 1) % 8)
28    children.append(Node(left_state, node, "turn_left",
29    node.depth + 1, node.cost + 1))
30
31    return children
```

The complete source code corresponding to this implementation can be found in the following public GitHub repository: https://github.com/claramingyue/AIF_PROYECT1